

MPI

viernes, 18 de septiembre de 2026 11:44

Al lanzar indicamos el número de procesos a usar.

Lo primero que debe haber al principio después del main es inicializar MPI, y al final hay que finalizar MPI.

FUNCIONES BÁSICAS MPI:

#include "mpi.h"

Inicializar mpi: ierr = MPI_Init(&argc, &argv);

Finalizar mpi: ierr = MPI_Finalize(); Es una barrera, así que si no terminan todos los procesos, mpi no termina

Compilar/ejecutar: mpicc / mpirun

Nº proc. Ejecutando: MPI_Comm_Size(MPI_COMM_WORLD, &nproces); &nproces es la variable en la que se retorna el número de procesos. MPI_COMM_WORLD es el comunicador que permite comunicar todos los procesos de un grupo de procesos

Averiguar id del proceso en ejecución: MPI_Comm_Rank(MPI_COMM_WORLD, &myrank); saca el id del proceso que ejecuta la instrucción y lo guarda en la variable myrank

Enviar: MPI_Send(&variable_enviar, num_datos, tipo_datos, destino, tag, comunicador); el tipo de datos es concreto de MPI_Datatype, como MPI_CHAR, MPI_INTEGER, MPI_DOUBLE... El tag debe ser igual en emisor y receptor

Recibir: MPI_Recv(&variable_recibir, num_datos, tipo_datos, origen, tag, comunicador, &status);

Broadcast: MPI_Bcast(&variable_enviar, num_datos, tipo_datos, id_raiz, MPI_COMM_WORLD). El id_propietario es el rank del proceso que hace el broadcast. El broadcast lo hace tanto el proceso que envía como los procesos que reciben.

```
MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
data1 = 0;
data2 = 0;
if(myrank > 0){
    err = MPI_Bcast(&data1, 1, MPI_INTEGER, 0, MPI_COMM_WORLD);
}
else{
    data1 = 33;
    err = MPI_Bcast(&data1, 1, MPI_INTEGER, 0, MPI_COMM_WORLD);
}
printf("Proceso %d. d1: %d - d2: %d.\n", myrank, data1, data2);
MPI_Finalize();
}
```

← Para los procesos que no son cero, recibe.

Hay que hacer broadcast SIEMPRE con el mismo nombre de la variable. Si no, se pensará que se hacen dos broadcast distintos.

En caso de tener que enviar a todos los procesos menos a uno, tengo que usar send/recv en lugar de broadcast o crear un grupo con todos los procesos menos en ese

Reducir: Es un broadcast inverso. Todos los procesos que NO son raíz envían datos al proceso raíz. El proceso que recibe no solo guarda los datos, sino que hace una operación con esos datos.

MPI_Reduce(&operando, &variable_resultado, num_datos, tipo_datos, operación, raíz, comunicador).

El proceso raíz también incluye sus datos.

Solo se usa una vez, no como en broadcast

Operaciones:

MPI_MAX
MPI_MIN
MPI_SUM
MPI_PROD
MPI_LAND
MPI_BAND
MPI_LOR
MPI BOR
MPI_LXOR
MPI_BXOR
MPI_MAXLOC (máximo y localización)*
MPI_MINLOC (máximo y localización)*

***Trabajan con pares de valores**

Para usar MINLOC o MAXLOC se debe generar una estructura:

```
struct {  
    double datos;  
    int rango;  
} est1[30], est2[30];
```

datos: que se comparan
rango: se asigna igual al rango

[6, 10, 20]
p1 = [1, 2, 3]
p2 = [4, 1, 2]
p3 = [1, 13, 25]
Des

A Reduce se le puede pasar vectores. Entonces, sumará columna a columna los vectores

ALLREDUCE: La función hace un reduce y luego el proceso raíz hace un broadcast del resultado. MPI_Allreduce(&operando, &resultado, num_datos, tipo_datos, operación, comunicador). Cada proceso recibe lo de todos los demás, hace la operación que se indica y lo guarda en su variable de resultado.

Sustituye el hacer un reduce y luego un broadcast del resultado

Recibir de cualquier emisor: se usa MPI_ANY_SOURCE, que sustituye el proceso del que vienen los datos. Ya no se comprueba de donde viene, solo de qué tamaño es el dato y qué etiqueta tiene.

Recibir de cualquier tag: MPI_ANY_TAG se usa en la recepción. Sustituye al tag en la sentencia de recepción

Ejemplo de envío y recepción de un escalar de un proceso dado (0) a otro (3)

```
int main(int argc, char *argv[])  
{  
    int nproces, myrank;  
    double data1=0, data2=0;  
    MPI_Status status;  
  
    MPI_Init(&argc, &argv);  
    MPI_Comm_size(MPI_COMM_WORLD, &nproces);  
    MPI_Comm_rank(MPI_COMM_WORLD, &myrank);  
    if (myrank == 0){  
        data1 = 4765.3;  
        MPI_Send(&data1, 1, MPI_DOUBLE, 3, 99, MPI_COMM_WORLD);  
    }  
    else{  
        if (myrank == 3)  
            MPI_Recv(&data2, 1, MPI_DOUBLE, 0, 99, MPI_COMM_WORLD, &status);  
    }  
    MPI_Finalize();  
}
```

La instrucción de enviar lo que hace es enviar el dato al buffer del proceso receptor.

El Finalize puede devolver que si el tamaño de datos a recibir es distinto al que se envía, hay datos en el buffer pendientes de recibir

Para hacer el código portable, el tamaño del mensaje que se envía y recibe deben ser los mismos

Un proceso de reducción es aquel en que un proceso 0 suma un entero que aportan todos los procesos. Los operandos son vectores y el resultado es un solo número. Esos vectores son los resultados parciales de los demás procesos